Rules Decide, Language Models Narrate: A Fail-Closed Multi-Agent Architecture for Flood Response with Numeric Grounding

Anonymous Author(s)
Affiliation withheld for double-blind review

***Abstract*—Large language models (LLMs) are proposed as the voice of emergency-management software, but they can state numbers that are not in the data. In our flood-response architecture, eleven agents share one state, a deterministic policy chooses every action and the LLM only narrates. Three mechanisms keep it fail-closed: a health-gated policy, a disagreement gate between two classifiers and a numeric guard that replaces any message with an unsupported number by a template. We evaluate all three on the deployed code. A composed check of the path from policy to dispatch found 2,807 of 32,032 cases in which teams were sent although the verified policy asked for review; after a one-line fix it found 0. Of 300 replies from two open-weight LLMs, 36.3% contained an unsupported number, almost all invented arrival times. The deployed guard caught 89.0% of these and a slot-typed guard 93.6%, with false-rejection rates of 5.8% and 10.5%; an LLM judge caught all it checked but needs a model call per message and missed 15% of injected one-digit probability errors. A revised guard, frozen before a third model family was queried, caught 98.7% of unsupported replies from that model, but its false rejections rose to 12.5%. On the classifiers' held-out data, held cases were wrong 1.8 times as often as released ones, and holding only when the models straddle the threshold held a quarter as many cases while flagging errors more sharply. Fail-closed properties must be checked on the whole path to action, and narration against typed state.**

***Index Terms*—large language models, hallucination, multi-agent systems, fail-safe design, flood response**

I. Introduction

During a flood, an emergency operations centre runs on short spoken messages: the river is at this level, the forecast says this, send a team there. Software that supports the centre is increasingly asked to *speak*, and LLMs make this easy. They also produce confident statements that their input does not support [1]. On an operations net a wrong number can trigger or cancel an evacuation.

Agentic systems let the model plan and call tools [2]–[4]. For a safety-critical setting we take the opposite position: decisions are made by small deterministic agents whose behaviour can be checked exhaustively, and the LLM only turns their state into radio traffic. This follows the view that the level of automation should be chosen per function and per risk [5] and that safety is a property of the control structure rather than of one component [6]. Two questions remain open. Does the verified decision actually govern what the system does? And how often does the narrator misstate the state in practice, and which check catches it?

We answer both on an open flood decision-support prototype for the Delhi Yamuna corridor. Our contributions are:

1) An architecture in which agents share one state and event log, only deterministic agents can act, and generated text is advisory, with three fail-closed mechanisms defined formally.
2) A composed check of the path from observation to dispatch, which found that dispatch bypassed the verified policy in 2,807 of 32,032 cases and confirmed the fix.
3) A measurement of numeric grounding on 300 replies of two open-weight LLMs and on injected errors, comparing the deployed guard, a slot-typed guard and an LLM-as-judge baseline [7], with a revised guard tested on a model family held out until the guard was frozen.
4) A held-out calibration study of the dual-model gate, which shows when model disagreement signals error and gives a tolerance set from training data alone.

II. Related Work

Agents and agentic LLMs. Software agents that perceive, decide and act have a long history [8], and disaster response has been a multi-agent benchmark since RoboCup Rescue [9]. LLM agents interleave reasoning with actions [2], learn when to call tools [3] and cooperate in conversations [4]; surveys describe them as memory, planner and action interface around a model [10]. Our agents keep memory and actions but take planning away from the model.

Hallucination detection and guardrails. Generated text departs from its source in many ways [1]. Retrieval grounds generation in documents [11]; post-hoc checkers decompose text into atomic claims [12], sample the model several times [13], let the model verify itself [14] or ask another model to judge [7]. Programmable rails constrain dialogue flows [15], runtime verification checks outputs against expert specifications [16], and runtime enforcement blocks unsafe agent actions [17]. These methods are probabilistic, rely on further model calls, or check actions rather than reported values. Our facts are a handful of numbers in a known state, so a deterministic, typed check is possible.

Safe automation. Levels of automation should fall where errors are costly [5]; run-time shields block unsafe actions of learned controllers [18]; and ensembles estimate uncertainty [19], [20]. We use disagreement between two classifiers as a hold signal instead of averaging them.

TABLE I
AGENTS, THEIR EVENTS AND THEIR AUTHORITY

Agent	Emits	Authority
Weather, Dam	rainfall, river and barrage stage	inform
Flood Risk	policy action π and reason	**decide** (gated)
Traffic, Shelter	road risk; ranked shelters	inform, rank
Evacuation	candidate routes, plan	propose
Rescue	team dispatch	act after π
Ambulance, Disaster Team	radio turns	narrate
Administrator	hold notices, radio turns	hold, narrate
Card Monitor	sudden-change alerts	inform

Algorithm 1 One pipeline tick (after the fix)

```
read observations; Weather and Dam agents emit updates
p_RF, p_XGB, p ← classifiers; H ← (2)
(h_d, h_m, h_s) ← health checks; a ← π(p, h, o) by (1)
if H and not o then a ← REVIEW and log the hold reason
if a ≠ MONITOR then rank roads, shelters, routes
if a ∈ {AUTO, OVERRIDE} and band(p) = auto then dispatch
T ← LLM turns for band(p); if not Γ(T) then T ← template
broadcast state, decision and T
```

Research gap. We found no study that measures, on the deployed code of an emergency system, both whether a verified decision governs actuation and how often real LLM narration misstates numeric state, nor one that compares deterministic and model-based checks of that narration.

III. ARCHITECTURE

Fig. 1 shows the system. A FastAPI backend runs one pipeline per tick, in live mode (Open-Meteo weather and river data) or in simulation mode (a digital twin of the corridor). Eleven agents share one state object and write every event to a MongoDB log and a WebSocket stream, so the interface shows what each agent did and why. Table I lists their authority: only the Flood Risk agent's policy can start a response, and dispatch executes only what the policy allowed. Algorithm 1 gives one tick. The policy and the gate run before any planning agent, and narration runs last, after the decision is logged, so a narration failure can delay a radio message but cannot change a decision.

A. Decision Policy

The Flood Risk agent receives a flood probability p and three health flags: h_d (weather and river data available), h_m (model loaded and returned a value) and h_s (database connected). An operator can set an override o, and a flag A enables automation. With a monitor threshold $\tau_m = 0.50$ and an automation threshold $\tau_a = 0.60$, served from one configuration module,

$$\pi(p, \mathbf{h}, o) = \begin{cases} \text{UNAVAILABLE} & p \text{ undefined} \\ \text{OVERRIDE} & o \\ \text{MONITOR} & p < \tau_m \\ \text{AUTO} & p \geq \tau_a \wedge h_d \wedge h_m \wedge h_s \wedge A \\ \text{REVIEW} & \text{otherwise.} \end{cases} \quad (1)$$

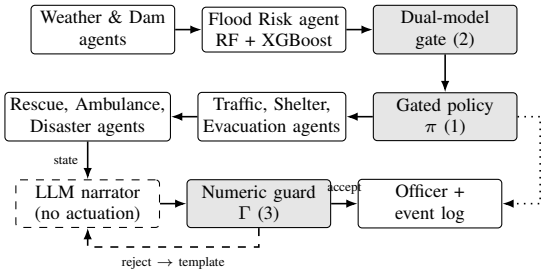


Fig. 1. Agent architecture. Shaded blocks can only hold, downgrade or reject; none can trigger an action. The LLM (dashed) receives the shared state and has no actuation rights. The dotted link carries every policy decision and its reason to the officer.

B. Dual-Model Disagreement Gate

The same features are scored by a Random Forest [21] and an XGBoost classifier [22]. Automatic dispatch is held when

$$H = \big[\, |p_{\mathrm{RF}} - p_{\mathrm{XGB}}| \geq \delta \,\big] \vee \big[\, \min(p_{\mathrm{RF}}, p_{\mathrm{XGB}}) < \tau_a \leq \max(p_{\mathrm{RF}}, p_{\mathrm{XGB}}) \big], \quad (2)$$

with $\delta = 0.15$. The first term flags a large gap; the second, the *straddle* term, flags the case where the models fall on opposite sides of the threshold. On a hold the Administrator agent announces it and the Rescue agent stays staged until the models agree or an operator overrides.

C. LLM Narration and the Numeric Grounding Guard

Agent conversations are generated by an LLM through an external API. The band follows the probability (calm below 0.40, watch below 0.50, administrator review below 0.60, automated response above), and a disagreement hold forces the administrator band; the band chooses who speaks and the tone, while the action is still set by π. The prompt lists the current facts (probability, rainfall, river level, barrage stage, model scores, time and place), states the band's rule and instructs the model to use only those numbers. The reply must be JSON with speaker, addressee and text.

Let $\mathcal{N}(s)$ be the decimal numbers in a string s and F the prompt text up to the recent-history block. With the admissible set $\tilde{F} = \{f, 100f, f/100 : f \in \mathcal{N}(F)\}$, a set of turns T is accepted when

$$\Gamma(T) = \bigwedge_{t \in T} \bigwedge_{n \in \mathcal{N}(t)} \exists f \in \tilde{F} : \; |n - f| \leq \tfrac{1}{2} 10^{-d(n)}, \quad (3)$$

where $d(n)$ is the number of decimals written in n, so honest rounding passes. A rejected reply is replaced by a template filled from the same state. The guard is untyped: a hallucinated value drawn uniformly from an integer range $[a, b]$ passes with probability

$$P_{\mathrm{miss}} \approx |\{\lfloor f \rceil : f \in \tilde{F}\} \cap [a, b]| \,/\, (b - a + 1), \quad (4)$$

which grows with the fact sheet, and numbers written as words are invisible to $\mathcal{N}$.

Slot-typed guard (prototype). Γ_{T} reads facts from the structured state and binds each number to a quantity before

checking it. Number words are first rewritten as digits; clock times must equal the current time; a number with "%" or "percent" must match a model probability or a threshold of the current band, one with "mm" the rainfall, and one with "m" the river level or barrage stage. A number that cannot be bound to a slot is rejected:

$$\Gamma_{\mathrm{T}}(T) = \bigwedge_{t \in T} \bigwedge_{(n,k) \in \mathcal{B}(t)} \exists f \in F_k : \ |n - f| \le \tfrac{1}{2} 10^{-d(n)}, \quad (5)$$

where $\mathcal{B}(t)$ pairs each number with its slot k and F_k holds the facts of that slot.

IV. Evaluation Setup

Policy and dispatch path. We evaluated (1) on $p \in \{0, 0.001, \ldots, 1\}$ plus an undefined value, all eight health combinations and both override states (16,032 cases), against five properties: AUTO only if every gate passes and $p \ge \tau_a$; undefined p gives UNAVAILABLE; $p < \tau_m$ without override gives MONITOR; $p \ge \tau_a$ with a failed check gives REVIEW; and with all checks passing the action never decreases in p. We then extracted the dispatch decision into a pure function and checked the composed path (policy, hold and dispatch) over all defined probabilities, health flags, holds and overrides (32,032 cases) for the property that teams are dispatched only when every check passes and the models agree, or when an operator overrides, with the original and the corrected rule.

Injected errors. We drew 200 seeded states (rainfall 5–250 mm, a monsoon month, both model probabilities from the deployed classifiers, river 200.0–208.5 m and barrage 199.0–206.0 m), and for each called the production function that builds the prompt, queries the API, parses the reply and applies Γ, with the HTTP client replaced by a stub returning a prepared reply: one grounded reply and seven copies with one wrong value of a given type (Table III).

Live replies. For the first 150 of these states we sent the production prompt to gpt-oss-20b and gpt-oss-120b on Groq at temperature 0.85, without cross-model fallback, and stored every reply (300 replies, all parsed) with the verdicts of Γ and Γ_{T}. Reference labels follow a written protocol. A number is *supported* if it states a value of the state for the quantity it describes (honest rounding allowed) or quotes a threshold of the band; *procedural* numbers (reporting intervals, priority labels, the count of named places, band ranges, deadlines given as orders) are not claims about the situation and are not counted; anything else, including any arrival time, which the state never contains, is *unsupported*. The protocol was applied mechanically, and every reply on which it disagreed with any checker, every reply it flagged and a random sample of the rest were then read in full; 4 labels were changed by hand.

LLM judge. As a model-based baseline, Qwen3 (27B), a model family different from the generators, received the structured facts and the turns, was told that honest rounding is allowed, and returned whether every number was supported [7]. It ran at temperature 0 on the first 40 injected states (320 replies) and on 241 live replies (all 150 of gpt-oss-120b and the first 91 of gpt-oss-20b), when the provider's daily token limit for the judge model stopped the run; a call that failed after retries counted as a rejection, as in the deployed system.

Revised guard and a held-out model family. We used the misses and false rejections on the 300 development replies to write a second typed guard, $\Gamma_{\mathrm{T}}^{(2)}$. It knows procedural numbers (reporting intervals, deadlines given as orders), rejects every arrival time whether it is written in digits, in words or as a clock time, and binds a value written without a unit to the quantity named just before it. We then froze the file, recorded its SHA-256 hash, and only afterwards collected and read replies from a third model family, ALLaM-2 (7B), for the same 150 states and prompt. 132 replies parsed; the other 18 calls failed or returned invalid JSON. We labelled them with the unchanged protocol and changed 8 labels by hand. The judge's daily quota was spent, so it did not see them.

Classifier calibration. The rainfall sweep used in the prototype varies two inputs with all others at defaults. We therefore rebuilt the stratified 80/20 split on which both deployed classifiers were trained (it reproduces their stored test metrics) and, on its 910 held-out rows, measured the expected calibration error (ECE, ten bins), the hold rate of (2) and the error rate of the policy model on held and released rows. A data-driven tolerance δ_{95} was set as the 95th percentile of the model gap in five-fold cross-validated predictions on the training split only.

V. Results

A. Policy and the Path to Dispatch

None of the 16,032 cases violated the five properties (the check took 42 ms), as expected for a five-line function. The composed check, however, found a fault. The original dispatch step tested the probability band and the disagreement hold but never read the action returned by π, so when a health check failed while both models agreed on a high probability, π correctly returned REVIEW and teams were still dispatched; the hold also applied only in simulation mode. With the original rule the check found 2,807 violating cases out of 32,032, all high-probability states with a failed check. After the fix (dispatch requires AUTO or OVERRIDE, and the hold applies in both modes) it found 0. Dispatches fell from 9,624 to 6,817, so the fix removed exactly the unsafe ones, and three regression tests now guard the rule.

B. Numeric Grounding of Live Replies

Table II gives the live results. 109 of the 300 replies (36.3%; 33.3% for gpt-oss-20b and 39.3% for gpt-oss-120b) stated at least one unsupported number, and 97% of those were invented arrival times such as "ETA 10 minutes", although the prompt contains no ETA. Every other value, including probabilities, rainfall, water levels and times, was almost always quoted correctly.

The deployed guard missed 12 unsupported replies. Two mechanisms explain most misses: arrival times written in words ("ETA about ten minutes"), invisible to $\mathcal{N}$, and arrival times given as a clock time equal to the current time, which match a real fact. Invented durations in digits were usually caught, unless they collided with another fact as (4) predicts.

TABLE II
CHECKS ON 300 LIVE LLM REPLIES (REFERENCE LABELS)

Check	all 300 replies Recall (95% CI)	FR	241 judged replies Recall	FR	Prec.
Deployed Γ	89.0 (82–94)	5.8	88.9	4.6	92.0
Typed Γ_{T}	93.6 (87–97)	10.5	94.4	9.9	85.0
LLM judge	–	–	100.0	10.6	84.9

All in %. Recall over unsupported replies (109 of 300; 90 of 241); FR = false rejections of supported replies; Prec. = precision of rejections.

TABLE III
DETECTION RATE (%) BY INJECTED ERROR TYPE

Injected error	200 states Γ	200 states facts only	200 states Γ_{T}	40 states Γ	40 states Γ_{T}	40 states judge
Grounded (false rej.)	0.0	0.0	0.0	0.0	0.0	0.0
Probability $\pm$1–5	94.0	95.0	97.0	97.5	100.0	85.0
Probability $\pm$10–30	95.5	96.5	97.0	92.5	95.0	97.5
Rainfall 0.5–1.5$\times$	97.0	97.5	100.0	95.0	100.0	100.0
River $\pm$0.5–2 m	98.0	98.0	98.5	100.0	100.0	100.0
Invented head-count	97.5	98.0	100.0	100.0	100.0	100.0
Invented ETA	94.0	95.0	100.0	95.0	100.0	100.0
Wrong value in words	0.0	0.0	81.5	0.0	92.5	90.0
All numeric errors	96.0	96.7	98.8	96.7	99.2	97.1

facts only: Γ without the band-rule numbers. Grounded row: false rejections.

The typed guard bound such numbers to no slot and caught more of them, but it rejected 20 supported replies, mostly for procedural numbers or values written without a unit ("barrage 202.26"). Those forms never occurred in our injected replies. The deployed guard's 11 false rejections were procedural numbers such as "report every 5 minutes". On the 241 replies it checked, the LLM judge caught every unsupported reply (recall 100.0%, against 88.9% and 94.4% for the two guards on the same replies) at a false-rejection rate of 10.6%, close to the typed guard's 9.9%. It reads a message as a whole, so it recognises an invented ETA in any wording, but it costs one model call per message and is itself a model whose verdicts cannot be checked exhaustively.

Finally, 43 replies (14%), all in the automated band, asserted that a flood "is confirmed" or that water "has breached", although the state holds only a probability, and no numeric check can see such claims.

C. Injected Errors

Table III compares the checks on injected errors. On all 200 states, the deployed guard rejected 96.0% of wrong digit strings with no false rejections and none of the wrong values written in words; the typed guard rejected 98.8% and 81.5% of number words. The 48 deployed misses were all collisions: 39 coincided with a genuine fact used for a different quantity (for example, with a probability of 87.3% an invented "ETA 87 minutes" passed), 8 with a threshold quoted in the band rule and 1 with a $\times 100$ copy. On the 40-state subset, the LLM judge detected 97.1% of numeric errors and 90.0% of number words; it was the weakest check on small probability errors, which change a single digit.

TABLE IV
GUARDS ON DEVELOPMENT REPLIES AND ON A HELD-OUT MODEL FAMILY

Check	gpt-oss, 300 (dev.) Recall	FR	ALLaM-2, 132 (held out) Recall (95% CI)	FR
Deployed Γ	89.0	5.8	96.1 (89–99)	7.1
Typed Γ_{T}	93.6	10.5	100.0 (95–100)	17.9
Typed v2 (frozen)	100.0	0.0	98.7 (93–100)	12.5

All in %. $\Gamma_{\mathrm{T}}^{(2)}$ was frozen by hash before any ALLaM-2 reply was read.

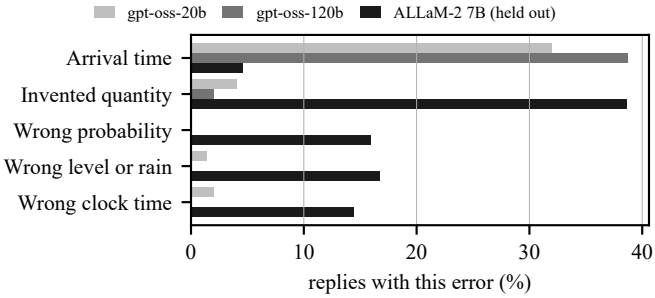


Fig. 2. Kinds of unsupported numbers in live replies, as a share of each model's replies (reference labels). A reply can contain more than one kind.

D. A Held-Out Model Family

The development replies came from one model family, and the revised guard was written after reading them, so its results on those replies say little. Table IV therefore separates the development set from the frozen test on ALLaM-2. On the development replies $\Gamma_{\mathrm{T}}^{(2)}$ reached a recall of 100.0% with 0.0% false rejections, as expected for a check tuned on them.

The held-out model behaved very differently from the development models. 76 of its 132 replies (57.6%) contained an unsupported number, and most of these were not arrival times but invented quantities: head-counts of affected residents, rates of rise, rain durations, coordinates and casualty estimates (64% of the unsupported replies). Fig. 2 shows how sharp the shift is: invented arrival times appeared in 32–39% of the gpt-oss replies but in only 4.5% of the ALLaM-2 replies, which instead invented quantities in 39% and misquoted a probability in 16%, an error the development models never made.

The frozen guard caught 98.7% of them and missed 1, against 96.1% for the deployed guard. Its false-rejection rate, however, rose from 0.0% to 12.5% (7 replies). Every one of these was a procedural number that the development models never wrote, such as "Team 1", "Mission 4" or "level 2 alert". The deployed guard rejected fewer grounded replies (7.1%), and the first typed guard more (17.9%). Table V shows four real excerpts.

Two lessons follow. First, a guard that is perfect on the replies used to build it can still fail on a new model, so a guard needs a held-out test just as a classifier does. Second, the errors of a typed guard on new traffic were safe ones: it replaced a correct message by a template, which costs some wording, while it let through only one unsupported number.

TABLE V
EXCERPTS FROM HELD-OUT REPLIES AND THE VERDICT OF EACH CHECK

Excerpt (content)	Label	Γ	Γ_{T}	$\Gamma_{\mathrm{T}}^{(2)}$
"probability of flooding is at 34.3%" (state value)	yes	pass	pass	pass
"begun flooding with 134 affected" (head-count)	no	**rej.**	**rej.**	**rej.**
"NDRF teams is 20:59 IST" (arrival time)	no	**rej.**	**rej.**	**rej.**
"Team 1 (Rescue) will head to" (label)	yes	**rej.**	**rej.**	**rej.**

Label: supported by the state (reference protocol).
rej.: the reply is replaced by a template.

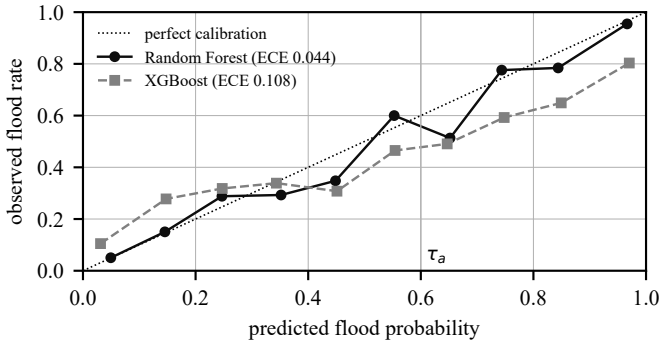


Fig. 3. Reliability of the two deployed classifiers on the 910 held-out rows (bins with at least 10 rows). Points below the diagonal are over-confident.

Checking one reply with $\Gamma_{\mathrm{T}}^{(2)}$ took 371 μs on average on a laptop, a negligible delay for a radio message.

E. Dual-Model Gate

On the prototype's rainfall sweep (604 settings) the gate held dispatch in 69.7% of settings, including 73.7% of those where both models already exceeded τ_a; the mean gap between the models (0.167) exceeded δ itself. Table VI explains why with held-out data. The XGBoost policy model was poorly calibrated (ECE 0.108, against 0.044 for the Random Forest; Fig. 3): above τ_a it predicted 0.84 on average where floods occurred at a rate of 0.67, so the two models differ systematically. Disagreement still carried information: on the 910 held-out rows the deployed gate held 26.9% of cases, and the policy model was wrong on 37.1% of held cases against 20.3% of released ones. The straddle rule alone held only 7.6% of cases, never one in which both models recommended action, and the policy model was wrong on 52.2% of them. The tolerance set from training data alone, $\delta_{95} = 0.28$, behaved similarly and is close to the value at which the sweep stopped holding cases on which both models agreed. Isotonic calibration on the training split lowered ECE to 0.018 and 0.031 and the hold rate of the deployed rule to 10.5%.

TABLE VI
GATE (2) ON 910 HELD-OUT ROWS OF THE CLASSIFIERS' SPLIT

Rule	Held %	Held, both $\geq \tau_a$ %	Error % held	Error % released
$\delta = 0.15$ (deployed)	26.9	29.1	37.1	20.3
$\delta_{95} = 0.28$ (train CV)	9.1	2.2	48.2	22.5
straddle only	7.6	0.0	52.2	22.6
isotonic, $\delta = 0.15$	10.5	7.0	39.6	22.4

Flood rate in the test rows: 35.8%. Error: of the policy model.
isotonic: both models calibrated on the training split.

VI. DISCUSSION AND LIMITATIONS

The results support a division of labour, and they locate its weak points in the glue between components. A dispatch step bypassed a verified policy, a gate tolerance was never related to the calibration of the models it compares, and an untyped guard treated every number as interchangeable. None of these showed up when components were tested one at a time. For safety-critical agentic systems this argues for checking properties on the full path from observation to action and for keeping the components with authority to act few and small. The live study changes the picture from injected errors, and the held-out model changes it again. The two development models rarely corrupted the facts they were given; they *added* quantities the state does not contain, above all arrival times. The held-out model added head-counts and rates and also misquoted the facts themselves. Which errors appear depends on the model, so a check designed around one model's habits has to be tested again on the next one. A check that only asks whether a number exists in the fact sheet is weakest on exactly that error, while a typed check needs rules for the procedural numbers that real traffic contains. A fail-closed fallback also hides what it absorbs: while we prepared this study, retired API models and a token budget too small for reasoning models had silently sent every reply to the template.

Several limits apply. The live study covers three open-weight models from two families, one prompt and 150 states; the held-out model already erred differently from the others, and further models will too. Reference labels were assigned with a written protocol and checked by reading, by one annotator; the distinction between claims and procedural numbers is a judgement, and we release the protocol and labels so that it can be re-examined. River and barrage values were sampled from operating ranges, not recorded events. The guards check numbers only, so the qualitative over-claims above, a wrong place or a wrong instruction would pass. The policy check proves properties of (1), not of the health checks that feed it, and the composed check covers the dispatch rule on a grid rather than a proof of the asynchronous pipeline. The calibration study uses the classifiers' random split of flood events; a time-aware split may give different numbers. No officer has used the system in an exercise.

For a deployment we draw three practical rules from these results. Ask the narrator not to produce arrival times, counts or unit labels, since the state contains none of them. Measure the template-fallback rate per model, because it is the only sign of a guard that has started to reject correct traffic. And re-run a labelled held-out test whenever the model behind the API changes, as providers retire and replace models without

notice.

Future work will deploy the typed guard with explicit procedural slots, forbid arrival times in the prompt and the guard, replace the gap tolerance with the straddle rule, and run a table-top exercise with emergency officers.

VII. Conclusion

We presented a flood-response architecture in which deterministic agents hold every decision and an LLM only narrates. Exhaustive testing verified the decision policy, yet the dispatch path did not consult it in 2,807 cases; after a one-line fix, the composed check found no violation in 32,032 cases. In 300 real replies of two open-weight LLMs, 36.3% stated an unsupported number, mostly invented arrival times; the deployed guard caught 89.0% of them and a typed guard 93.6%. A revised guard kept 98.7% recall on a held-out model but rejected 12.5% of correct replies, while an LLM judge caught all it checked but missed one-digit errors that the typed guard caught. On held-out data, model disagreement did signal error, and the straddle rule held a quarter as many cases as the deployed gate while flagging errors more sharply. LLM narration can serve an operations net only if it stays outside the decision path and is checked, with types, against the state it describes.

Acknowledgment

The authors thank the INDOFLOODS dataset authors and Open-Meteo for openly available data. In line with the IEEE policy on AI-generated content, the authors disclose that Claude (Anthropic) was used to help edit and restructure the text of all sections and to draft the experiment and plotting scripts. All experiments use the authors' own software and can be reproduced with fixed seeds; the live LLM replies are stored with the scripts. The authors checked every number, claim and reference and take full responsibility for the content.

References

[1] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023, doi: 10.1145/3571730.

[2] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[3] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Adv. Neural Inf. Process. Syst. 36*, 2023.

[4] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[5] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[6] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. Cambridge, MA, USA: MIT Press, 2011.

[7] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Adv. Neural Inf. Process. Syst. 36 (Datasets Benchmarks Track)*, 2023.

[8] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *Knowl. Eng. Rev.*, vol. 10, no. 2, pp. 115–152, 1995, doi: 10.1017/S0269888900008122.

[9] H. Kitano *et al.*, "RoboCup Rescue: Search and rescue in large-scale disasters as a domain for autonomous agents research," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Tokyo, Japan, 1999, pp. 739–743, doi: 10.1109/ICSMC.1999.816643.

[10] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, Art. no. 186345, 2024, doi: 10.1007/s11704-024-40231-1.

[11] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. 33*, 2020, pp. 9459–9474.

[12] S. Min *et al.*, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 12076–12100, doi: 10.18653/v1/2023.emnlp-main.741.

[13] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2023, pp. 9004–9017, doi: 10.18653/v1/2023.emnlp-main.557.

[14] S. Dhuliawala *et al.*, "Chain-of-verification reduces hallucination in large language models," in *Findings Assoc. Comput. Linguist. (ACL)*, 2024, pp. 3563–3578, doi: 10.18653/v1/2024.findings-acl.212.

[15] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. Conf. Empir. Methods Nat. Lang. Process.: Syst. Demonstrations*, 2023, pp. 431–445, doi: 10.18653/v1/2023.emnlp-demo.40.

[16] Y. Zhang, S. Y. Emma, A. L. J. En, and J. S. Dong, "RvLLM: LLM runtime verification with domain knowledge," in *Adv. Neural Inf. Process. Syst. 38*, 2025.

[17] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[18] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018, pp. 2669–2678, doi: 10.1609/aaai.v32i1.11797.

[19] T. G. Dietterich, "Ensemble methods in machine learning," in *Multiple Classifier Systems* (LNCS, vol. 1857). Berlin, Germany: Springer, 2000, pp. 1–15, doi: 10.1007/3-540-45014-9_1.

[20] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Adv. Neural Inf. Process. Syst. 30*, 2017, pp. 6402–6413.

[21] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[22] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.